FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Yago Felipe de Oliveira Martins

Produto: **Caravana's Hub**
https://github.com/YagoMartins77/Caravanas-Hub

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 2
3.1 Visão geral, modelo de negócio e componentes 2
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 5
3.5 Modelo de dados do domínio 7
3.6 Segurança 8
3.7 Observabilidade, decisões técnicas, limitações e próximos passos 8
3.8 Camadas IaaS, PaaS e SaaS do projeto 9
4 Quadro de entregas 9
5 Atividades realizadas 10
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 10
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 12
5.3 Atividade 9 — Fork e plano de negócio 13
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 14
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 18
6 Considerações finais 20
7 Declaração de autoria 20

1 Introdução

A disciplina **Introdução à Computação em Nuvem** (ISW055) conduziu, ao longo do 2º bimestre, a evolução de uma aplicação web acadêmica em containers Docker até um produto **Software as a Service** (SaaS) completo em produção, com autenticação, controle de acesso por papéis (RBAC), trilha de auditoria, armazenamento de objetos e monetização recorrente integrada ao Stripe.

No 1º bimestre, a base técnica do projeto consistia em um catálogo de filmes do ator Tom Hanks (`Tom_Hanks`), utilizado para exercitar a containerização com múltiplos serviços, autenticação com banco relacional, envio de e-mail transacional e registro de auditoria em Redis Streams. A partir da Atividade 9, esse histórico técnico foi migrado para um repositório próprio e transformado no **Caravana's Hub**, um produto real voltado à organização e venda de pacotes completos de viagem em caravanas esportivas.

A dor resolvida pelo **Caravana's Hub** é a dificuldade enfrentada por torcedores do interior paulista — com foco inicial em Marília, Alta Paulista e Centro-Oeste Paulista — para ir a jogos do Brasileirão, corridas de Fórmula 1 (Interlagos) e eventos de MMA em São Paulo sem precisar negociar transporte e ingresso separadamente. A plataforma reúne em um único pacote a vaga na van ou ônibus (com cálculo transparente de custos) e o ingresso a preço de custo, operando nos planos **Free** e **Pro**.

Este relatório consolida a arquitetura efetivamente em execução no servidor da disciplina (`yago-martins-isw055.lapps.studio`), apresenta a modelagem de dados, os fluxos de requisição e assinatura, as camadas de nuvem (IaaS, PaaS e SaaS), os controles de segurança e observabilidade, o quadro de entregas e as fichas detalhadas das atividades 7 a 11.

2 Metodologia

As atividades 7 e 8 foram desenvolvidas no repositório público original do 1º bimestre (`https://github.com/YagoMartins77/Tom_Hanks`). Na atividade 9, esse repositório teve todo o histórico espelhado (`git clone --bare` seguido de `git push --mirror`) para o repositório definitivo do produto (`https://github.com/YagoMartins77/Caravanas-Hub`), onde o domínio de caravanas esportivas e os recursos SaaS completos foram implementados nas atividades 10 e 11, utilizando a mesma stack no Portainer e o mesmo domínio público (`https://yago-martins-isw055.lapps.studio`).

As datas e horários realizados registrados no quadro de entregas e nas fichas individuais foram conferidos diretamente no histórico de commits (`git log`, fuso `UTC-3`), nas páginas dos commits no GitHub e, a partir da atividade 9, na abertura das issues de entrega mencionando o professor (`@siriani`).

As evidências visuais foram coletadas diretamente da interface publicada em produção, do console do navegador para validação dos bloqueios HTTP `403`, do MinIO Console, dos logs gravados no Redis via `log-service`, do painel da stack no Portainer e do Dashboard do Stripe em modo de teste. Os diagramas de arquitetura, fluxo de assinatura e modelo de dados foram gerados especificamente a partir do código-fonte e do arquivo `docker-compose.yml` vigentes.

3 Arquitetura do SaaS

3.1 Visão geral, modelo de negócio e componentes

O **Caravana's Hub** atende dois perfis operacionais principais e uma administração centralizada: **passageiros** (usuários finais que buscam e reservam caravanas), **organizadores** (responsáveis pela frota de vans/ônibus que publicam viagens e fazem upload de fotos dos veículos) e **administradores** (responsáveis

por cadastrar eventos oficiais, criar cupons de desconto, promover organizadores, realizar sorteios de brindes e auditar logs).

O modelo de receita adota o formato **freemium** combinado com reserva de passagens via Stripe Checkout em modo de teste:

- **Plano Free:** gratuito; permite até 2 reservas ativas por mês (`LIMITE_RESERVAS_MES_FREE`), sem acesso antecipado à pré-venda, sem aplicação de cupons promocionais e sem participação automática nos sorteios de brindes.
- **Plano Pro (R$ 19,90/mês):** assinatura recorrente via Stripe; libera reservas ilimitadas no mês, pré-venda com antecedência configurada em `ANTECEDENCIA_PRO_DIAS = 3` dias antes da abertura para o plano Free, uso de cupons de desconto na compra da passagem e elegibilidade para sorteios de brindes nas caravanas.

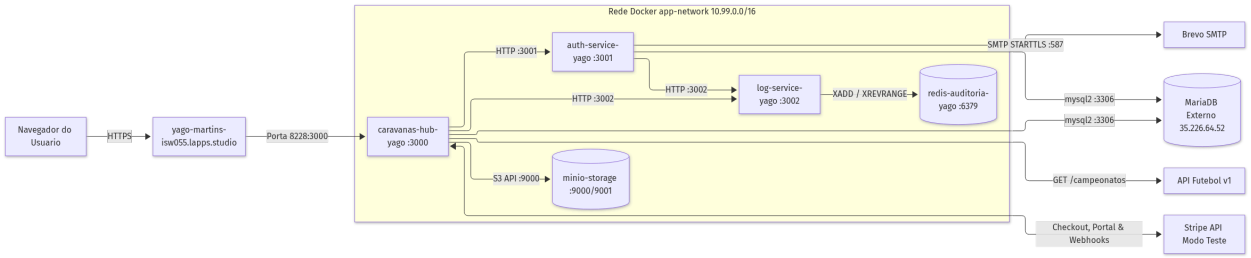


Figura 1: Diagrama de componentes e microsserviços em execução no Caravana's Hub

Componente	Responsabilidade	Tecnologia / imagem	Público?
`caravanas-hub-yago`	Landing page, frontend (`app.html`, `auth.html`, `reset-password.html`), API do domínio, proxy de auth/logs, RBAC, upload MinIO e integração Stripe.	Node.js 20 (`node:20-alpine`), Express 4.19	sim (`8228:3000` via domínio)
`auth-service-yago`	Cadastro blindado contra Mass Assignment, login com `bcrypt` (regra `F2` para admin) e envio de e-mail de redefinição de senha.	Node.js 20 (`node:20-alpine`), Express, Nodemailer	não (rede interna `:3001`)
`log-service-yago`	Recebe eventos de negócio/auditoria via HTTP interno e grava/consulta/remove entradas no Stream `auditoria:logs`.	Node.js (`node:20-alpine`), Express 4.18, `redis`	não (rede interna `:3002`)
`redis-auditoria-yago`	Armazenamento persistente dos eventos de auditoria e logs de negócio em Redis Streams.	`redis:alpine` vol. `redis_auditoria_data`	não (rede interna `:6379`)
`minio-storage`	Object Storage compatível com S3 para armazenamento das imagens dos veículos publicadas pelos organizadores.	`minio/minio:latest` vol. `minio_data`	sim (`9003:9000` e `9004:9001`)
MariaDB externo	Persistência relacional do domínio (`contas`, `caravanas`, `reservas`, `cupons`, planos Free/Pro, `sessions` e `reset_tokens`).	MariaDB externo `35.226.64.52:3306`	não (acesso via IP da VPS)

Componente	Responsabilidade	Tecnologia / imagem	Público?
Stripe (externo)	Checkout de assinatura Pro (`mode: subscription`), Checkout de passagens, Billing Portal e disparo de webhooks assinados.	Serviço SaaS (`stripe` SDK v23)	webhook `/api/stripe/webhook`
API Futebol e Brevo	Consulta de clubes/escudos do Brasileirão (`api.api-futebol.com.br/v1`) e disparo SMTP STARTTLS (`smtp-relay.brevo.com:587`).	APIs externas SaaS	saída HTTP / SMTP

3.2 Onde roda

A aplicação está hospedada na VM da disciplina (gerenciada pelo professor na infraestrutura da Fatec), orquestrada via **Docker Compose** dentro de uma stack no **Portainer** vinculada ao repositório `https://github.com/YagoMartins77/Caravanas-Hub`. O tráfego HTTPS público chega pelo túnel Cloudflare configurado para `https://yago-martins-isw055.lapps.studio`, encaminhando requisições para a porta `8228` do host (mapeada para a porta `3000` do container `caravanas-hub-yago`).

Todos os 5 containers compartilham a rede bridge `app-network` com sub-rede configurada para `10.99.0.0/16`:

- `caravanas-hub-yago` (porta externa `8228:3000`), `auth-service-yago` (porta interna `3001`), `log-service-yago` (porta interna `3002`), `redis-auditoria-yago` (porta interna `6379`, volume `redis_auditoria_data`) e `minio-storage` (portas `9003:9000` e `9004:9001`, volume `minio_data`).
- Na infraestrutura do Portainer da disciplina aplica-se teto de 1 GB de memória por container; no arquivo `docker-compose.yml` do repositório não foi declarada diretiva explícita `mem_limit`.

Os segredos nunca são versionados no Git: ficam configurados nas variáveis de ambiente da stack no Portainer e documentados apenas pelos **nomes** no arquivo `.env.example`:

- **Aplicação e rede:** `PORT`, `APP_URL`, `SESSION_SECRET`, `AUTH_SERVICE_URL`, `LOG_SERVICE_URL`, `REDIS_URL`.
- **Banco de dados externo:** `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`.
- **Regras SaaS e Stripe:** `STRIPE_PUBLISHABLE_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_ID`, `ANTECEDENCIA_PRO_DIAS`, `LIMITE_RESERVAS_MES_FREE`.
- **Storage e serviços externos:** `MINIO_ENDPOINT`, `MINIO_PORT`, `MINIO_PORT_PUBLIC`, `MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`, `API_FUTEBOL_KEY`, `SMTP_USER`, `SMTP_PASS`.

3.3 Fluxo de uma requisição

Tomando como exemplo a ação **"passageiro reserva uma vaga em caravana"** (`POST /api/reservas`):

1. O navegador do usuário envia a requisição HTTPS para `https://yago-martins-isw055.lapps.studio/api/reservas` carregando o cookie `HttpOnly` da sessão.
2. O Cloudflare Tunnel encaminha a chamada para `caravanas-hub-yago:3000`.
3. O middleware `express-session` (backed pelo `express-mysql-session` na tabela `sessions` do MariaDB) restaura o ID da conta autenticada; caso não haja sessão válida, retorna `401`.
4. O middleware `carregarUsuario` consulta as tabelas `contas` e `usuarios_premium` no MariaDB para obter o papel atualizado (`passageiro`, `organizador` ou `admin`) e confirmar se o usuário possui assinatura Pro ativa; em seguida, `exigirPapel('passageiro')` valida se a conta tem permissão para reservar (`403` caso seja outro papel).

5. A rota consulta a caravana desejada, verifica vagas disponíveis e aplica as travas de plano: se o usuário for Free e a data atual for anterior à janela de abertura pública (`abertura_vendas_em`), bloqueia com `403`; se já possuir 2 ou mais reservas ativas no mês (`LIMITE_RESERVAS_MES_FREE = 2`), registra auditoria e retorna `403` indicando upgrade para o plano Pro.
6. Estando elegível (e validando eventual cupom em `cupons` somente para assinantes Pro), o serviço cria a reserva ou inicia a sessão de pagamento da passagem no Stripe Checkout, envia o evento de auditoria ao `log-service-yago:3002` (que executa `XADD` no Redis `auditoria:logs`) e devolve a resposta JSON ao navegador.

3.4 Fluxo de assinatura

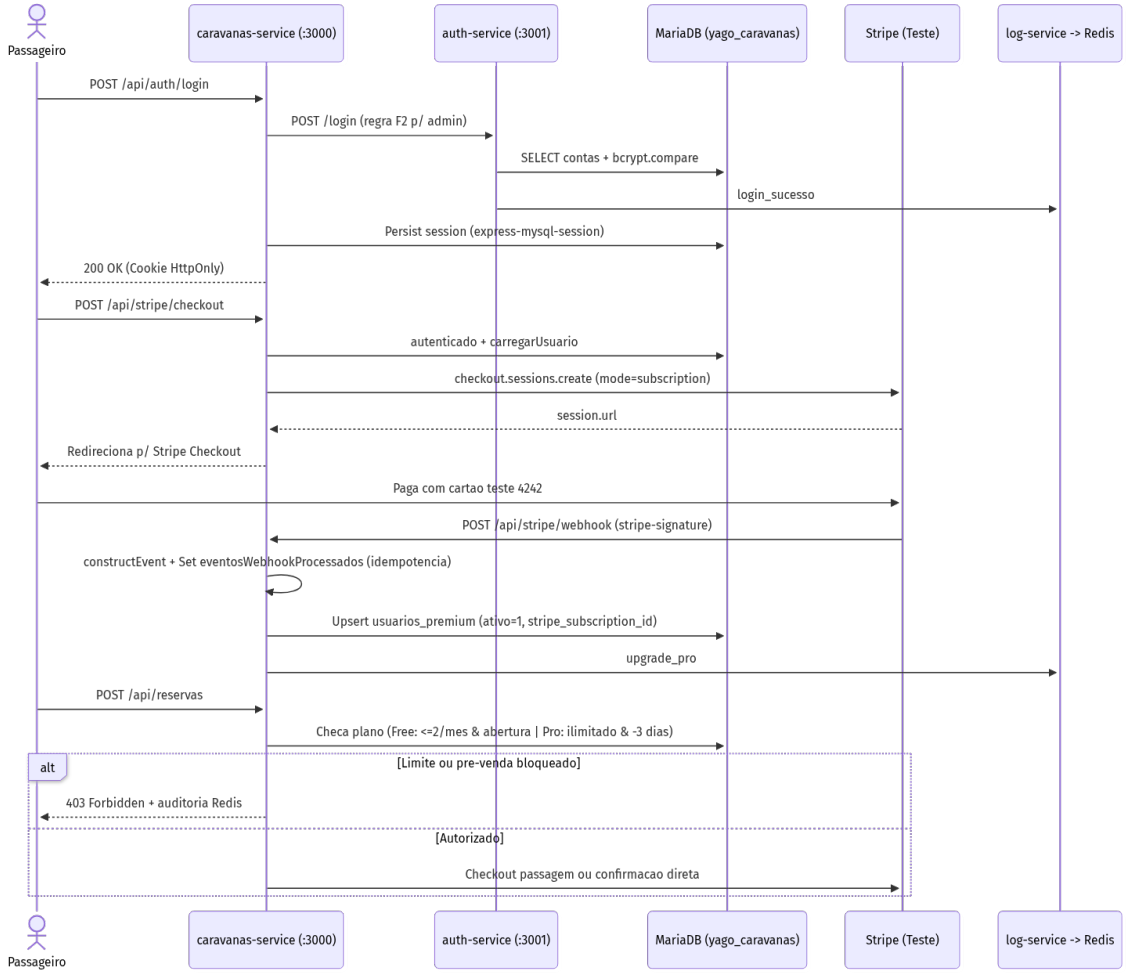


Figura 2: Fluxo de assinatura recorrente no Stripe, tratamento de webhooks com idempotência e aplicação de limites por plano

Quando o passageiro clica em **"Assinar Pro"** dentro do sistema, o frontend chama `POST /api/stripe/checkout`. O backend autentica a sessão, garante ou cria o cliente no Stripe (`obterOuCriarStripeCustomer`) e cria uma **Checkout Session** com `mode: 'subscription'`, `client_reference_id` com o ID da conta e `STRIPE_PRICE_ID` (ou `price_data` recorrente mensal de R$ 19,90), redirecionando o usuário para o checkout hospedado do Stripe em modo de teste.

Após o pagamento (ex.: cartão de teste `4242...`), o Stripe dispara eventos para `POST /api/stripe/webhook`, montado com `express.raw({ type: 'application/json' })` para preservar o corpo bruto exigido pela verificação criptográfica `stripe.webhooks.constructEvent(req.body, req.headers['stripe-signature'], STRIPE_WEBHOOK_SECRET)`. Caso a assinatura falhe, a rota rejeita

com HTTP `400`. Para evitar efeitos duplicados em retentativas do Stripe, a aplicação mantém o conjunto em memória `eventosWebhookProcessados` (retendo os últimos `2000` `event.id` processados) e utiliza inserção/atualização idempotente (`ON DUPLICATE KEY UPDATE`) na tabela `usuarios_premium`.

Quatro eventos são tratados no manipulador de webhook:

- `checkout.session.completed`: se `mode === 'subscription'`, executa `ativarPro(...)`, gravando `stripe_customer_id`, `stripe_subscription_id`, `ativo = 1` e `status_assinatura = 'active'` em `usuarios_premium` e auditando no Redis; se `metadata.tipo === 'passagem'`, confirma a reserva paga.
- `customer.subscription.updated`: sincroniza o `status` da assinatura (`active` ou `trialing` mantêm `ativo = 1`; outros estados rebaixam para `ativo = 0` e inserem registro em `usuarios_free`).
- `customer.subscription.deleted`: marca `ativo = 0` e `status_assinatura = 'canceled'` em `usuarios_premium` e garante a volta imediata do usuário para `usuarios_free`.
- `invoice.payment_failed`: diante de falha na cobrança recorrente, marca `status_assinatura = 'past_due'` e `ativo = 0`, impedindo que o assinante inadimplente continue usufruindo do Pro gratuitamente.

Para gestão de cobrança e cancelamento pelo próprio cliente, a aplicação expõe `POST /api/stripe/portal` (que gera sessão no **Stripe Billing Portal**), `POST /api/stripe/cancelar` (cancelamento direto na API do Stripe retornando a conta ao plano Free) e as rotas auxiliares `GET /api/stripe/verify-session` e `POST /api/stripe/sincronizar`.

3.5 Modelo de dados do domínio

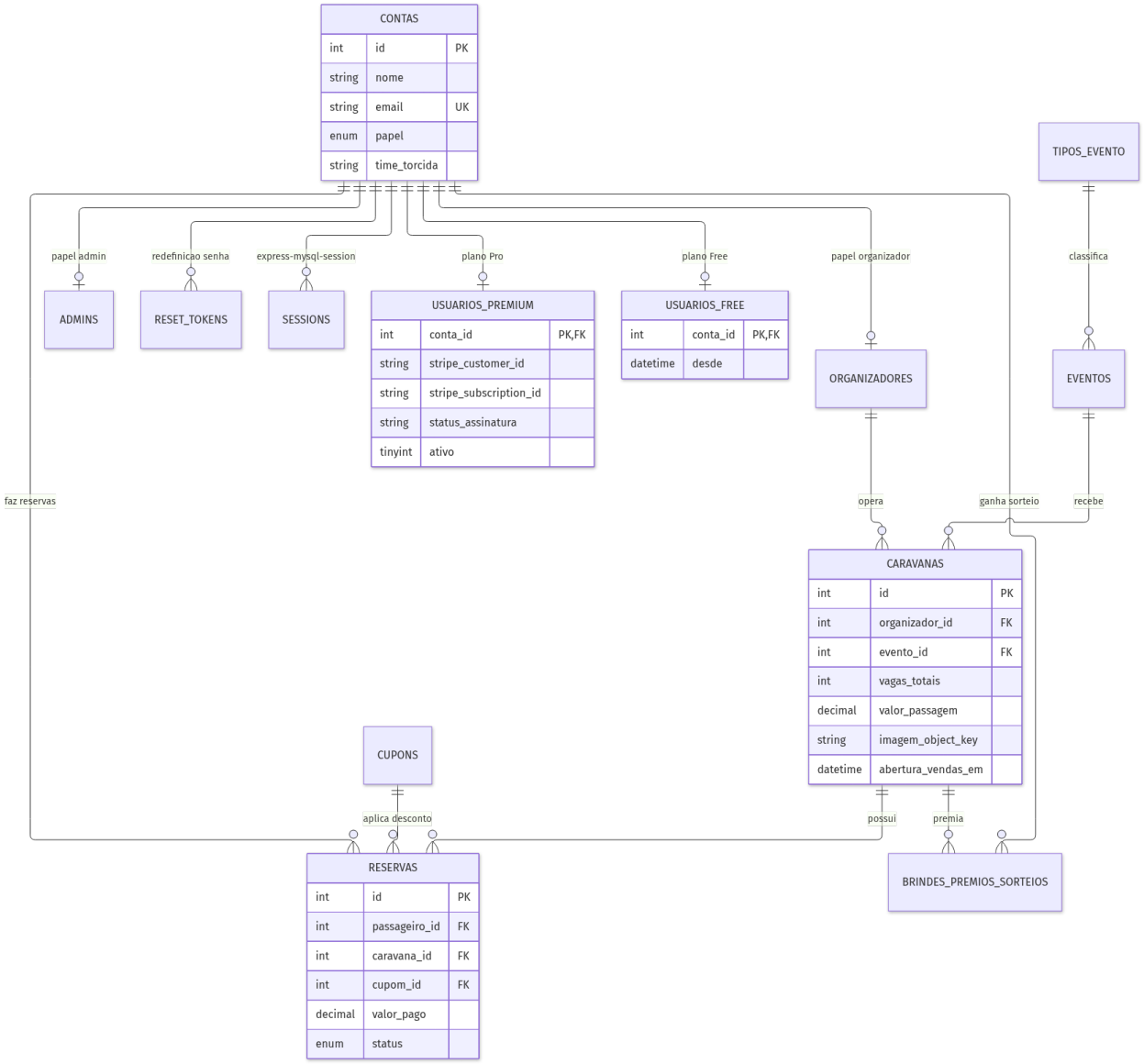


Figura 3: Diagrama entidade-relacionamento das tabelas de domínio, assinaturas e sessões no MariaDB

O banco relacional MariaDB (`db/001_dominio.sql` e migrações automáticas de inicialização em `caravanas-service/server.js`) organiza o sistema em torno de uma identidade única (`contas`) combinada com especializações por perfil e plano:

- **Identidade e planos:** `contas` (`id`, `nome`, `email`, `senha_hash`, `papel`, `time_torcida`), `admins` (`conta_id`, `nivel`, `setor`), `organizadores` (`conta_id`, `documento`, `descricao_frota`, `contato_whatsapp`), `usuarios_free` (`conta_id`, `desde`) e `usuarios_premium` (`conta_id`, `stripe_customer_id`, `stripe_subscription_id`, `status_assinatura`, `ativo`, `data_assinatura`).
- **Núcleo de caravanas e vendas:** `tipos_evento`, `eventos` (`tipo_id`, `titulo`, `data_evento`, `local_evento`), `caravanas` (`organizador_id`, `evento_id`, `cidade_saida`, `ponto_encontro`, `valor_passagem`, `vagas_totais`, `imagem_object_key`, `abertura_vendas_em`), `reservas` (`passageiro_id`, `caravana_id`, `cupom_id`, `valor_pago`, `status`, `stripe_session_id`), `cupons` (`codigo`, `percentual_desconto`, `ativo`, `validade`, `usos_maximos`) e `brindes_premios_sorteios` (`caravana_id`, `ganhador_free_id`, `ganhador_premium_id`, `descricao`).

- **Infraestrutura relacional, logs e objetos:** `sessions` (`express-mysql-session`), `reset_tokens` (tokens SHA/aleatórios com expiração de 30 minutos para recuperação de senha via Brevo), Redis Stream `auditoria:logs` e buckets `avatars` e `caravanas` no MinIO.

3.6 Segurança

Ação no sistema	admin	organizador	passageiro
Cadastrar tipos de evento e eventos oficiais	sim	não	não
Publicar caravanas e enviar fotos ao MinIO	sim	sim	não
Reservar vagas, usar cupons (Pro) e assinar Pro	não	não	sim
Criar/alterar cupons e realizar sorteios de brindes	sim	não	não
Consultar/filtrar/excluir logs de auditoria no Redis	sim	não	não
Promover conta de passageiro para organizador	sim	não	não
Ver/editar dados de outro usuário ou outra frota	sim (admin)	não	não

O isolamento entre usuários ocorre em todas as consultas autenticadas: rotas de passageiros filtram explicitamente por `WHERE passageiro_id = req.usuario.id` (e só permitem cancelar a própria reserva), enquanto rotas de organizadores filtram por `WHERE organizador_id = req.usuario.id`, impedindo alteração ou visualização de reservas e frotas de terceiros (retornando HTTP `403` ou `404` quando outro ID é informado). O papel (`papel`) é reconsultado no MariaDB em cada requisição pelo middleware `carregarUsuario` — não ficando preso apenas em cookie/sessão estática.

Entre os controles adicionais implementados estão: hash de senhas com `bcrypt` (fator de custo `12`), blindagem contra **Mass Assignment** na rota `POST /register` do `auth-service` (toda conta pública nasce obrigatoriamente como `passageiro` no plano Free), restrição de login administrativo apenas via atalho `F2` (`via_atalho_f2`) no `auth-service`, cabeçalhos de segurança com `helmet` e CSP, limitação de taxa com `express-rate-limit` e verificação criptográfica `stripe-signature` no webhook. Todos os segredos ficam isolados nas variáveis de ambiente da stack do Portainer, mantendo apenas os nomes das chaves em `.env.example`.

3.7 Observabilidade, decisões técnicas, limitações e próximos passos

A observabilidade de negócio e segurança é implementada pelo microsserviço `log-service` (`log-service-yago:3002`), que grava eventos no Stream `auditoria:logs` do Redis via comando `XADD` e os expõe ordenados (`XREVRANGE`) para o painel administrativo (`GET /api/admin/logs`), incluindo exclusão controlada (`DELETE /api/admin/logs/:id`). Tanto o `auth-service` (`cadastro_sucesso`, `login_sucesso`, `login_falha`, `login_admin_bloqueado`, `solicitacao_recuperacao_senha`, `senha_redefinida`) quanto o `caravanas-service` (`caravana_criada`, `vaga_reservada`, `cupom_aplicado`, `cupom_gerado`, `cupom_lote_gerado`, `brinde_sorteado`, `upgrade_pro`, `assinatura_atualizada`, `assinatura_cancelada_webhook`, `pagamento_renovacao_falhou`, `portal_cliente_aberto`, `cancelar_pro`, `tentativa_negada_403`, `limite_free_atingido`, `limite_prevenda_pro`, `upload_avatar`, `foto_caravana_enviada`) enviam registros com usuário, ação, detalhes e IP de origem.

Não foram configurados neste projeto pipelines automáticos de CI/CD no GitHub Actions, especificação interativa Swagger/OpenAPI nem endpoints de métricas Prometheus/Grafana — o deploy atual ocorre via

atualização da stack no Portainer apontando para o repositório no GitHub, acompanhado pela documentação em `README.md`, `db/001_dominio.sql` e `.env.example`.

Entre as principais decisões técnicas e limitações atuais do SaaS estão:

1. **Idempotência de webhooks em memória:** o conjunto `eventosWebhookProcessados` protege contra duplicidades em RAM (até 2.000 eventos), sendo zerado em reinícios do container; embora as queries SQL usem atualizações idempotentes (`ON DUPLICATE KEY UPDATE`), o próximo passo ideal é persistir `event_id` em tabela dedicada no MariaDB.
2. **Dependência de credenciais externas em ambiente de teste:** Stripe opera com chaves `sk_test_`/`whsec_` e a entrega de e-mails de redefinição de senha depende de chave SMTP válida da Brevo configurada no Portainer.
3. **Ausência de suíte automatizada E2E e healthcheck profundo:** atualmente a validação das rotas é feita manualmente e por inspeção sintática; como evolução futura, recomenda-se adicionar testes automatizados de integração cobrindo webhooks do Stripe e controle RBAC, além de um endpoint `/health` verificando conectividade simultânea com MariaDB, Redis e MinIO.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no Caravana's Hub	Quem gerencia
IaaS	Servidor Linux (VPS), CPU, RAM, armazenamento em disco, rede virtual e IP público onde o servidor da disciplina é executado.	Professor / Infraestrutura Fatec
PaaS	Engine Docker, Portainer (orquestração da stack, rede `app-network` `10.99.0.0/16` e limites de 1 GB por container), instância gerenciada de MariaDB (`35.226.64.52`), Redis Streams (`redis:alpine`) e Object Storage MinIO (`minio/minio`).	Professor (Portainer/MariaDB) e Aluno (definição da stack no `docker-compose.yml` e buckets/streams)
SaaS	O próprio produto **Caravana's Hub** entregue aos passageiros e organizadores em `yago-martins-isw055.lapps.studio`, além dos serviços SaaS de terceiros consumidos pela aplicação: Stripe (pagamentos e assinaturas), GitHub (código e issues), Cloudflare Tunnel (DNS/TLS), Brevo (e-mail SMTP) e API Futebol (clubes/escudos).	Aluno (lógica, frontend e API do produto) e Fornecedores externos (Stripe, GitHub, Cloudflare, Brevo, API Futebol)

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	09/10/2026 16:33	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	24/10/2026 00:25	entregue com atraso

Nº	Atividade	Data planejada	Data realizada	Situação
9	Fork e plano de negócio	23/10/2026	11/11/2026 18:21	entregue com atraso
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	13/11/2026 18:57	entregue com atraso
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	13/11/2026 23:32	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	09/10/2026 16:33
Situação	entregue
Evidência	GitHub (Tom_Hanks) — commit e658b75, issue #6 (@siriani) e 3 prints do Stripe e banco
Link	https://github.com/YagoMartins77/Tom_Hanks/issues/6

O que foi feito. Ainda na base do catálogo de filmes (`Tom_Hanks`), foi implementado o fluxo inicial de assinatura do Plano Premium em modo de teste do Stripe. Foram criadas rotas no backend para gerar a **Checkout Session** do Stripe, receber o webhook oficial validando a assinatura criptográfica (`STRIPE_WEBHOOK_SECRET`) e atualizar o status do usuário no MariaDB para premium, liberando recursos exclusivos diretamente na interface.

As entregas de código ocorreram no dia 09/10/2026 nos commits `d82e893`, `742705c`, `76620ef` e no commit final `e658b755b8eacce1d8bae7d1925ae00d2b68f7d8` (`09/10/2026 16:33`), acompanhados da abertura da issue #6 no repositório Tom_Hanks às `16:34` (UTC-3).

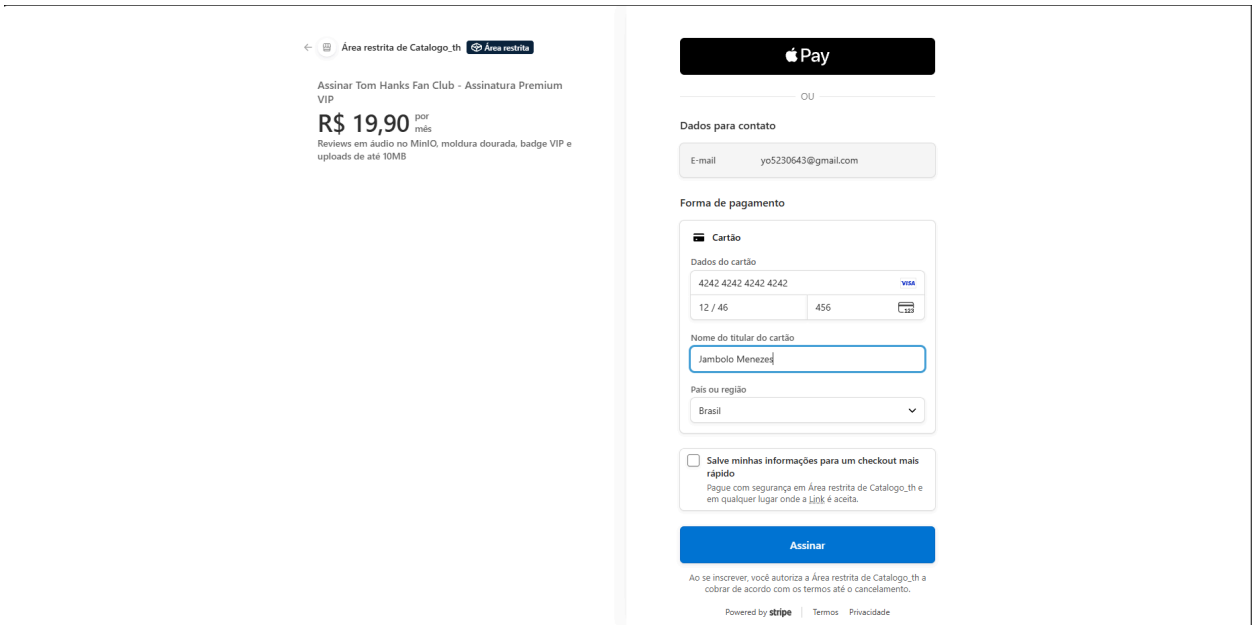


Figura 4: Atividade 7 — Checkout do Stripe em modo de teste concluído

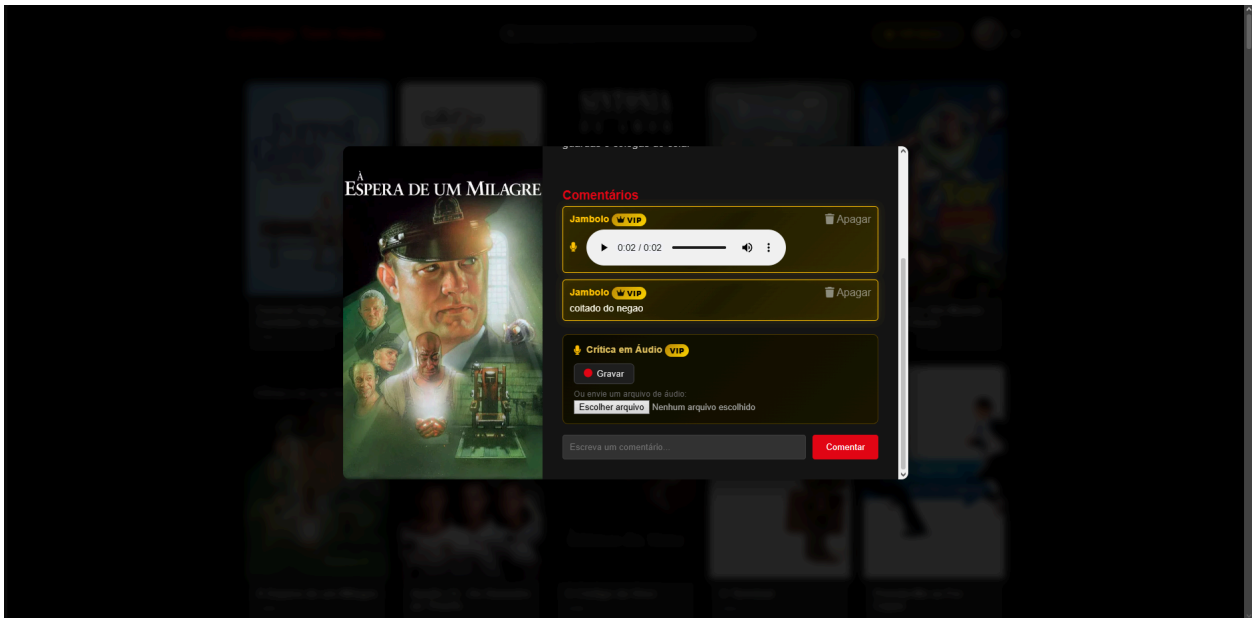


Figura 5: Atividade 7 — Benefício premium funcionando na interface da aplicação

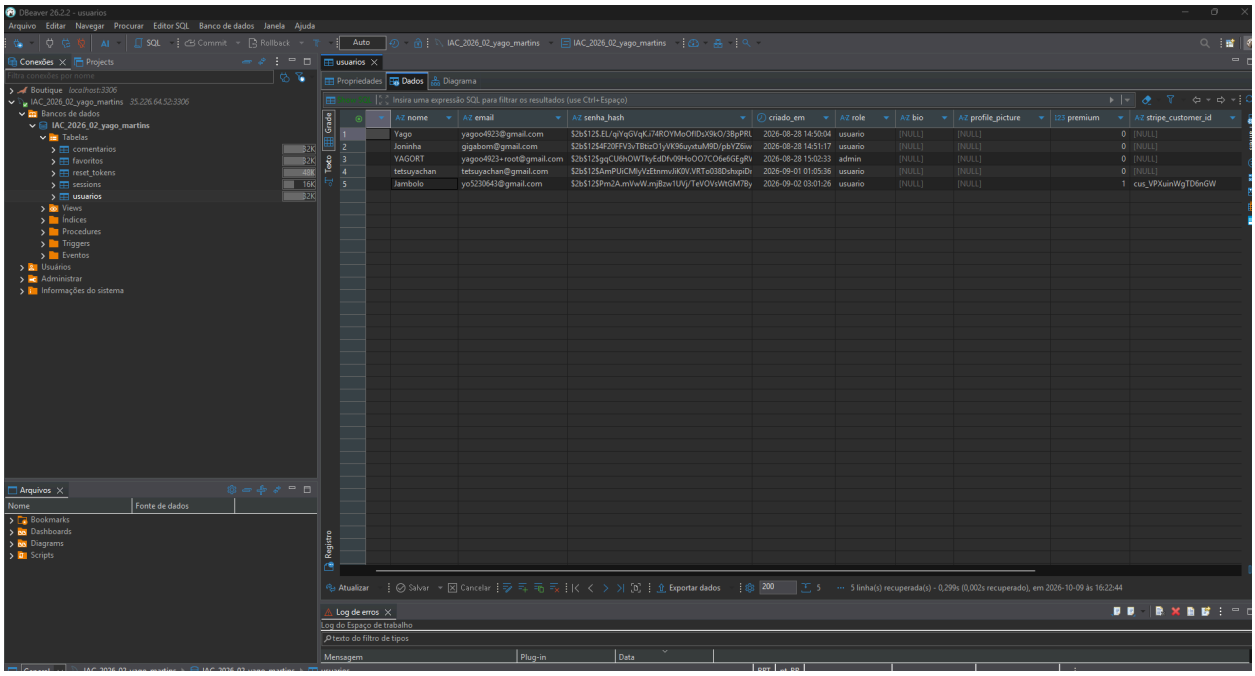


Figura 6: Atividade 7 — Registro do usuário marcado como premium no banco MariaDB após o webhook

Dificuldades e como foram resolvidas. A principal dificuldade foi validar corretamente a assinatura do webhook no Express usando o corpo bruto da requisição (`express.raw`) antes do parser JSON global, além de sincronizar a variável `STRIPE_WEBHOOK_SECRET` entre o painel do Stripe e o ambiente do container no Portainer. Após isolar a rota do webhook antes do `express.json()` e conferir os logs de entrega de eventos no Stripe, o fluxo passou a atualizar o MariaDB imediatamente.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	24/10/2026 00:25
Situação	entregue com atraso
Evidência	GitHub (Tom_Hanks) — commits 78b7e05 e 71dc4be, DOCUMENTACAO.md, README.md e issue #7
Link	https://github.com/YagoMartins77/Tom_Hanks/issues/7

Justificativa do atraso. A entrega prevista para `16/10/2026` foi concluída apenas em `24/10/2026` **por falha interna** na organização do cronograma e publicação da documentação no repositório.

O que foi feito. Foi realizada a engenharia reversa completa da aplicação existente no repositório `Tom_Hanks`, produzindo o arquivo `DOCUMENTACAO.md` e atualizando o `README.md` com o link direto para a documentação técnica. O material descreveu a arquitetura de microsserviços em Docker (`app`, `auth-service`, `log-service`, `redis`), a classificação detalhada das camadas IaaS, PaaS e SaaS aplicadas ao sistema em execução na VPS, o modelo de dados no MariaDB, as regras de negócio de autenticação/assinatura e a tabela completa de variáveis de ambiente.

Evidências publicadas diretamente no GitHub:

- Issue de entrega: github.com/YagoMartins77/Tom_Hanks/issues/7 (aberta em `24/10/2026 00:33` UTC-3);
- Commit final: `71dc4be` (`24/10/2026 00:25`) (e commit `78b7e05` em `00:01`);
- Arquivos documentais: `README.md` e `DOCUMENTACAO.md`.

Dificuldades e como foram resolvidas. O maior desafio foi levantar cada tabela, rota e dependência oculta no código que havia sido construído incrementalmente ao longo do bimestre e separá-las com clareza entre IaaS, PaaS e SaaS. A leitura direta dos arquivos `server.js`, `docker-compose.yml` e das configurações do Portainer permitiu fechar uma documentação fiel ao ambiente real em execução.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	11/11/2026 18:21
Situação	entregue com atraso
Evidência	GitHub (Caravanas-Hub) — issue #6 (@siriani), backlog (#1 a #5), README.md do produto e prints
Link	https://github.com/YagoMartins77/Caravanas-Hub/issues/6

Justificativa do atraso. A entrega prevista para `23/10/2026` foi realizada em `11/11/2026` **por falha interna** no planejamento da transição do projeto antigo para o novo produto.

O que foi feito. Todo o histórico Git do projeto `Tom_Hanks` foi preservado e espelhado (`git clone --bare` + `git push --mirror`) para o repositório definitivo do novo SaaS: `https://github.com/YagoMartins77/Caravanas-Hub`. Em seguida, o `README.md` foi completamente reescrito para definir o plano de negócio do **Caravana's Hub**: dor do mercado (dificuldade de torcedores de Marília e região organizarem transporte + ingresso para jogos, F1 e MMA em São Paulo), persona do passageiro e do organizador, **Lean Canvas**, modelo de receita freemium (**Free** vs **Pro R$ 19,90/mês**), arquitetura alvo e mapa de reaproveitamento entre a base anterior e o novo domínio.

Também foram abertas as 5 issues de backlog do produto (`#1` a `#5`) antes do registro da entrega formal na issue #6 (`11/11/2026 18:21` UTC-3).

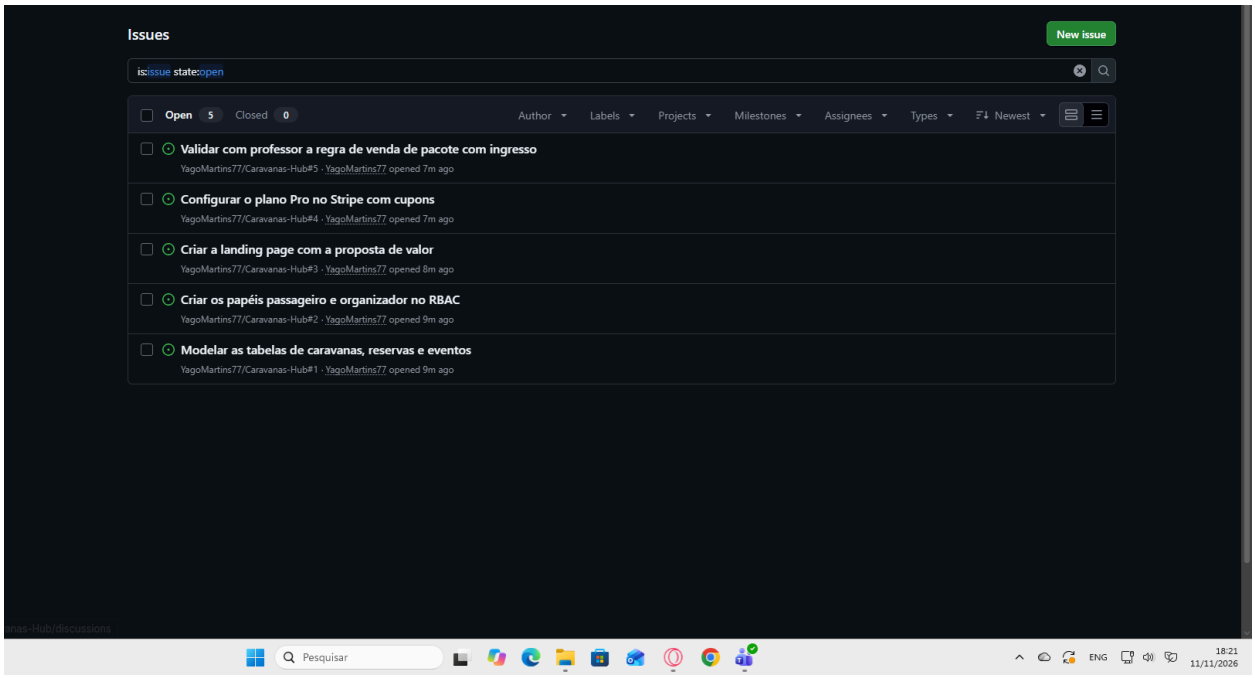


Figura 7: Atividade 9 — Backlog estruturado nas issues #1 a #6 do repositório Caravanas-Hub

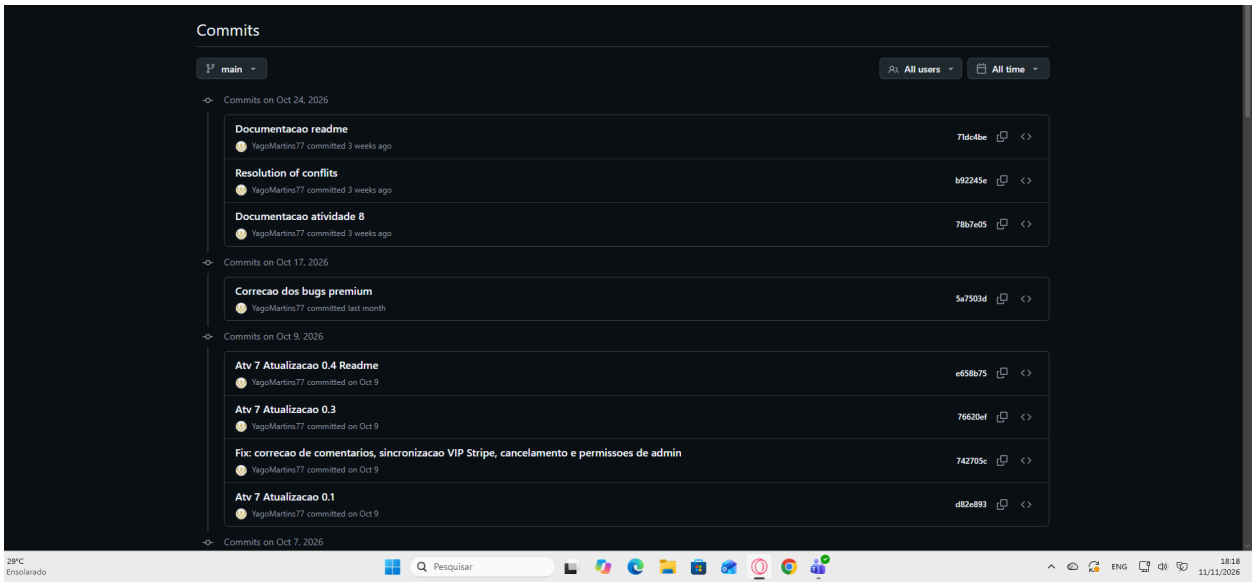


Figura 8: Atividade 9 — Histórico de commits preservado após o espelhamento do repositório

Dificuldades e como foram resolvidas. O ponto crítico foi executar o espelhamento preservando todos os commits das atividades 1 a 8 sem corromper referências nem perder o vínculo do projeto anterior, além de modelar os benefícios do plano Pro (antecedência, cupons e sorteios) de forma financeiramente viável no contexto de excursões esportivas. Testar o `git push --mirror` em um repositório recém-criado resolveu a migração do histórico de forma limpa.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	13/11/2026 18:57

Situação	entregue com atraso
Evidência	GitHub (Caravanas-Hub) — issue #7 (@siriani), URL no ar e 8 prints do sistema, RBAC 403, logs Redis, MinIO e Portainer
Link	https://github.com/YagoMartins77/Caravanas-Hub/issues/7

Justificativa do atraso. A entrega prevista para `06/11/2026` foi concluída em `13/11/2026` **por falha interna** durante a migração completa do domínio e ajustes de deploy em produção.

O que foi feito. O antigo catálogo de filmes foi integralmente substituído pelo domínio do **Caravana's Hub** em `https://yago-martins-isw055.lapps.studio`. A migração SQL (`db/001_dominio.sql`) removeu `favoritos` e `comentarios` e criou as tabelas `contas`, `admins`, `usuarios_free`, `usuarios_premium`, `organizadores`, `tipos_evento`, `eventos`, `caravanas`, `reservas`, `cupons` e `brindes_premios_sorteios`.

Na camada de aplicação (`caravanas-service` e `auth-service`), foram implementados: landing page comercial do produto, listagem de caravanas com cálculo transparente de custos e escudos via API Futebol, controle de acesso por papéis (`admin`, `organizador`, `passageiro`) retornando HTTP `403` com auditoria no Redis em tentativas não autorizadas, isolamento de consultas por `passageiro_id` e `organizador_id`, upload de fotos dos veículos no **MinIO**, eventos de negócio no `log-service` e deploy da stack atualizada no **Portainer**.

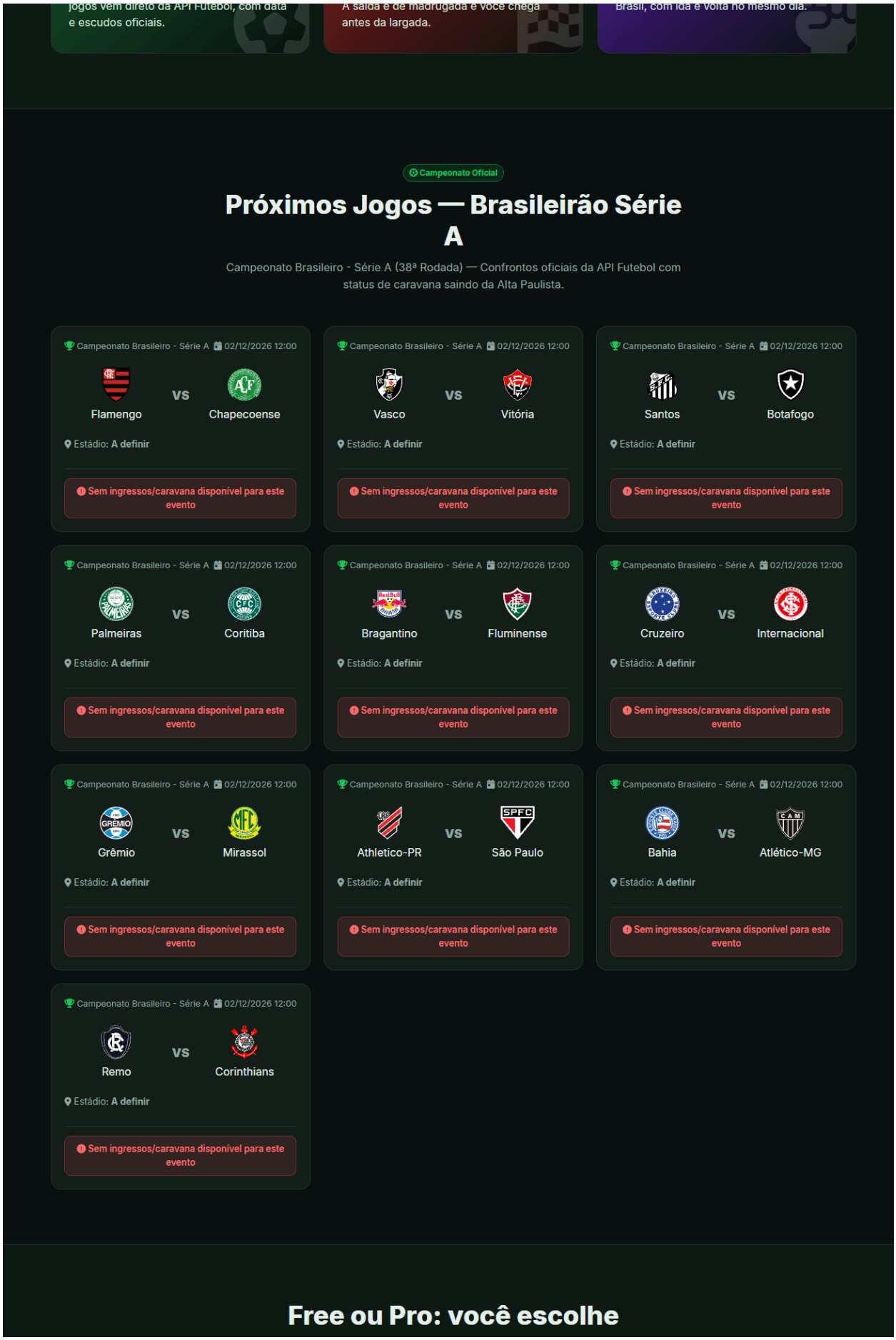


Figura 9: Atividade 10 — Landing page pública do Caravana's Hub no ar

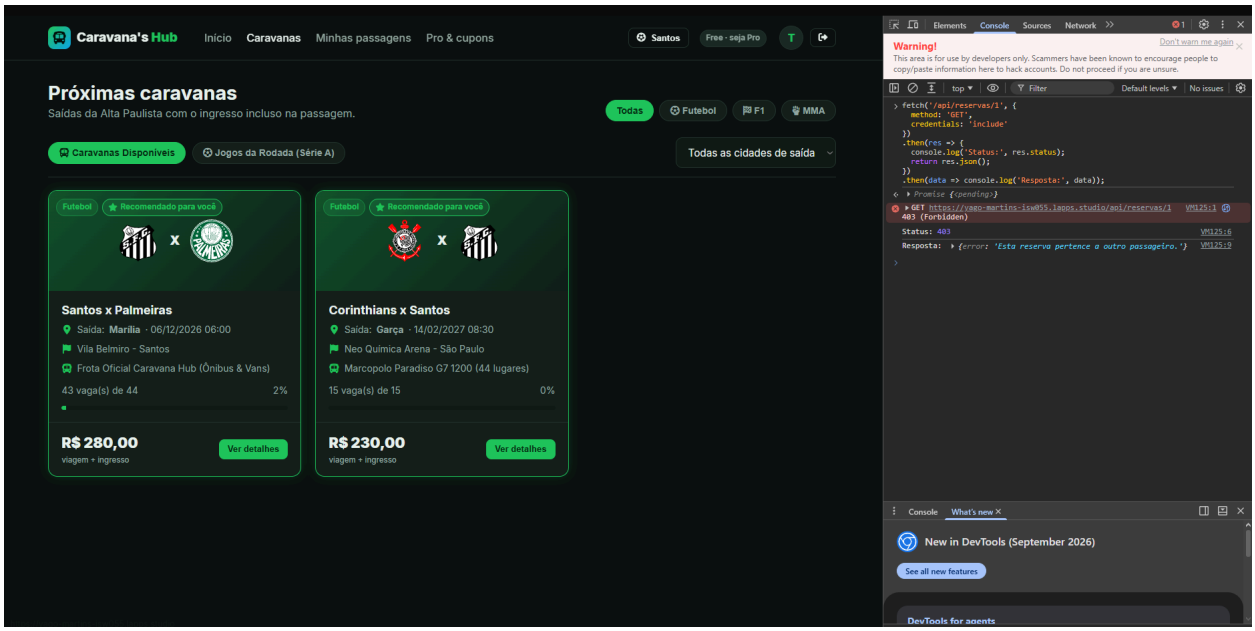


Figura 10: Atividade 10 — Listagem de caravanas com cálculo transparente e reserva

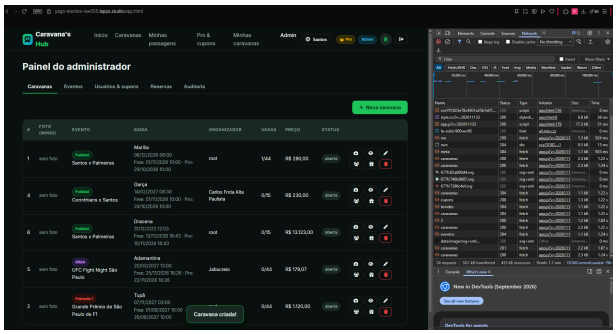


Figura 11: Atividade 10 — Bloqueio RBAC 403 (teste 1)

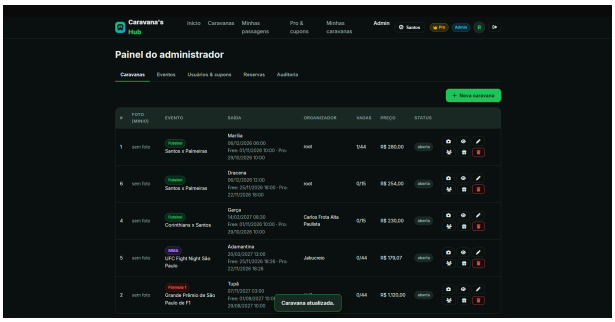


Figura 12: Atividade 10 — Bloqueio RBAC 403 (teste 2)

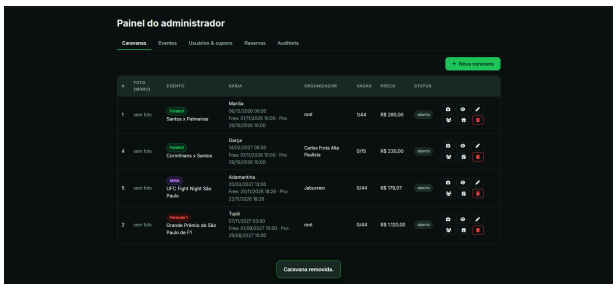


Figura 13: Atividade 10 — Bloqueio RBAC 403 (teste 3)

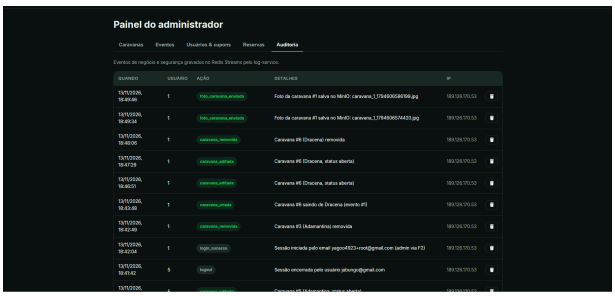


Figura 14: Atividade 10 — Eventos de negócio registrados no log (Redis)

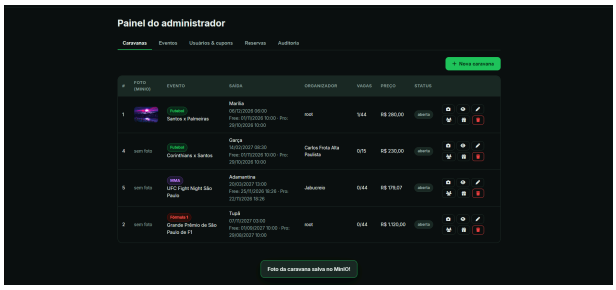


Figura 15: Atividade 10 — Imagem da caravana armazenada no MinIO

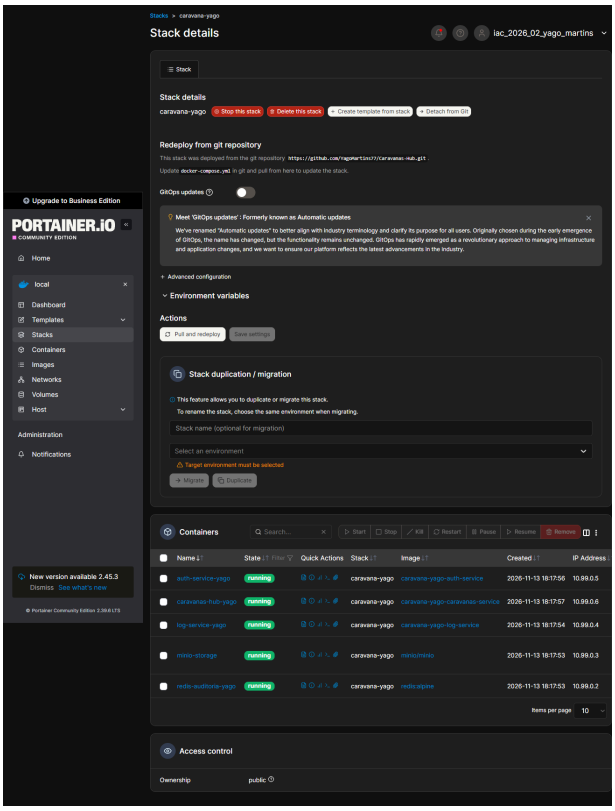


Figura 16: Atividade 10 — Stack no Portainer com os 5 containers saudáveis

Dificuldades e como foram resolvidas. Durante a transição das tabelas e do frontend surgiram erros de CSP no console, falhas no carregamento de alguns escudos de clubes e necessidade de proteger o atalho administrativo (F2) para que contas comuns não autenticassem como admin fora do fluxo esperado. A revisão da política `helmet` CSP, o fallback local dos escudos no servidor e a validação explícita de `via_atalho_f2` no `auth-service` eliminaram os bloqueios antes do deploy final.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	13/11/2026 23:32
Situação	entregue
Evidência	GitHub (Caravanas-Hub) — issue #8 (@siriani), commit 30aa716 e 5 prints do cartão 4242, Stripe, webhook, bloqueio 403 e admin
Link	https://github.com/YagoMartins77/Caravanas-Hub/issues/8

O que foi feito. Foi consolidado o ciclo SaaS completo de planos e assinatura recorrente no **Caravana's Hub**. O plano Pro (R$ 19,90/mês) ganhou fluxo completo de assinatura via Stripe Checkout (`mode: subscription`), validação de 4 eventos de webhook (`checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted` e `invoice.payment_failed`) com verificação de assinatura e idempotência por `event.id`, além de suporte ao **Stripe Billing Portal** e cancelamento retornando o usuário automaticamente para `usuarios_free`.

Na API de negócio, foram aplicadas as travas reais por plano: o usuário **Free** possui limite de 2 reservas ativas por mês (`LIMITE_RESERVAS_MES_FREE = 2`), bloqueando tentativas excedentes com HTTP `403`, e o assinante **Pro** passa a acessar pré-venda com 3 dias de antecedência (`ANTECEDENCIA_PRO_DIAS = 3`), cupons promocionais e sorteios de brindes. O painel administrativo também foi concluído com gestão de eventos, criação de cupons, realização de sorteios, métricas SaaS (Free vs Pro e receita estimada) e visualização/exclusão de logs do Redis. A entrega foi formalizada na issue #8 em `13/11/2026 23:32` (UTC-3).

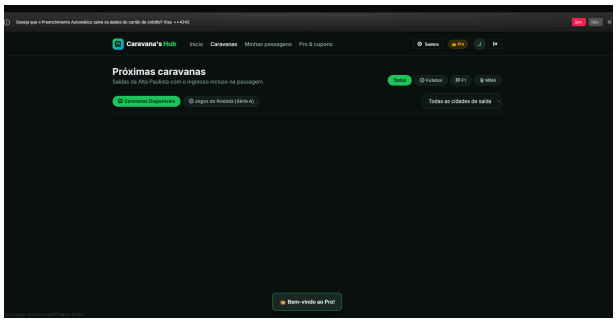


Figura 17: Atividade 11 — Assinatura Pro concluída com cartão de teste 4242

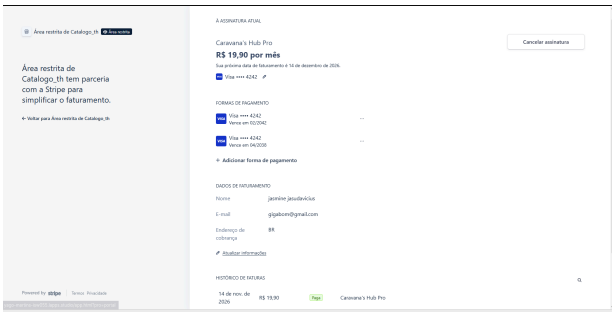


Figura 18: Atividade 11 — Transação registrada no Dashboard do Stripe (modo teste)

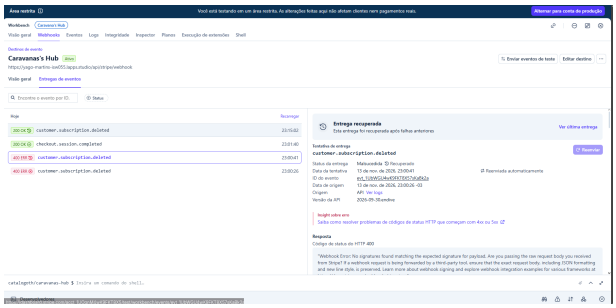


Figura 19: Atividade 11 — Webhook recebido e plano atualizado automaticamente

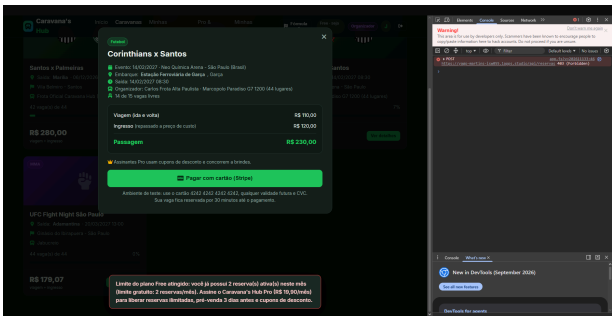


Figura 20: Atividade 11 — Limite do plano Free bloqueando reserva excedente (HTTP 403)

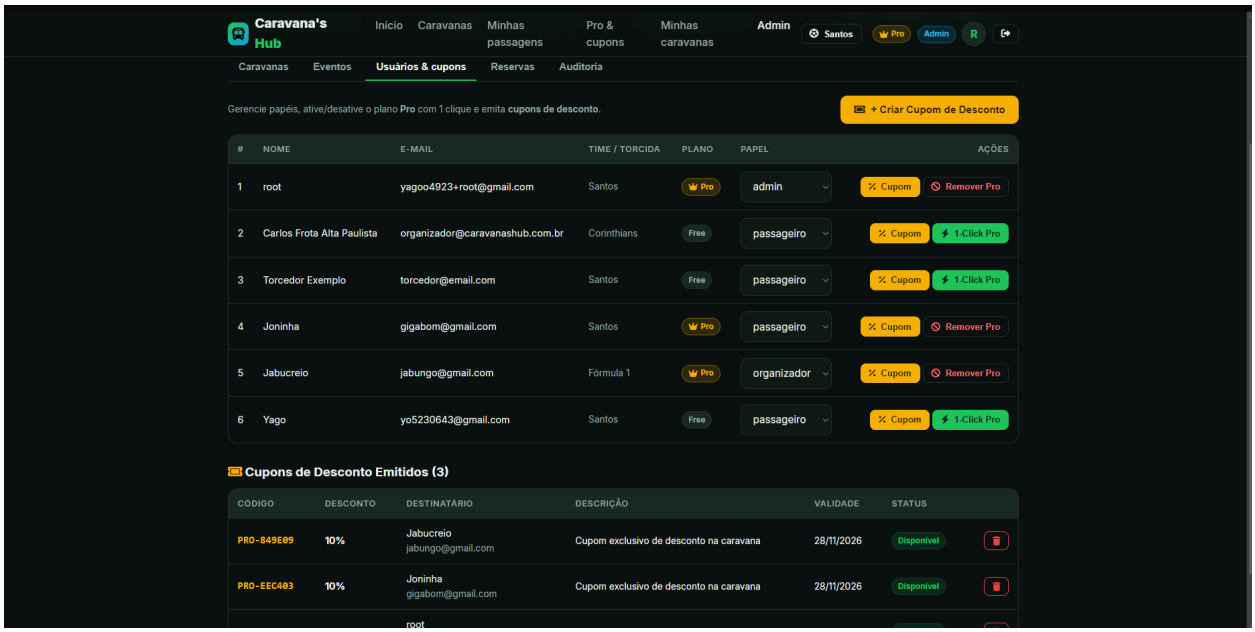


Figura 21: Atividade 11 — Painel administrativo com gestão SaaS, cupons, sorteios e auditoria

Dificuldades e como foram resolvidas. O principal desafio técnico foi garantir a consistência do ciclo completo entre assinatura recorrente, compra de passagem com cupom de desconto no Stripe Checkout,

cancelamento do plano Pro e bloqueio automático tanto na criação da reserva gratuita acima do limite quanto em tentativas de compra antes da abertura para o plano Free. A unificação dos helpers de verificação de plano (`carregarUsuario`) e a gravação automática do evento no Redis em cada bloqueio (`limite_free_atingido`, `limite_prevenda_pro`) tornaram os testes transparentes e auditáveis.

6 Considerações finais

Ao longo do 2º bimestre, a evolução do repositório `Tom_Hanks` para o **Caravana's Hub** demonstrou na prática como decisões de arquitetura em nuvem impactam diretamente a confiabilidade de um SaaS. Mais do que colocar páginas no ar, o projeto exigiu integrar múltiplos containers isolados em rede interna Docker (`auth-service`, `log-service`, Redis e MinIO), trabalhar com persistência externa no MariaDB, validar webhooks criptografados do Stripe sem confiar apenas no retorno do navegador e aplicar regras de negócio e autorização diretamente no servidor.

Os pontos mais desafiadores foram reorganizar o esquema relacional completo na migração do domínio mantendo a integridade de sessões e identidades, ajustar políticas de segurança (`helmet`, cookies `HttpOnly`, proteção contra **Mass Assignment** e regra administrativa por atalho `F2`) sem quebrar os fluxos legítimos de frontend, e lidar com o sincronismo assíncrono dos eventos do Stripe em ambiente de produção via túnel Cloudflare. As entregas intermediárias tiveram atrasos pontuais por falha interna de planejamento, mas todas as funcionalidades centrais, evidências e travas exigidas foram concluídas e verificadas no ambiente publicado.

Se o projeto fosse reiniciado ou levado agora para um ambiente comercial real, os próximos passos prioritários seriam persistir a tabela de idempotência de webhooks no próprio MariaDB (em vez de cache apenas em memória), implementar uma suíte de testes automatizados de integração cobrindo RBAC e pagamentos antes de cada deploy e estruturar um pipeline CI/CD no GitHub Actions com endpoint `/health` monitorado para eliminar atualizações manuais na stack do Portainer.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Yago Felipe de Oliveira Martins

Pompeia, 13/11/2026